USENIX OSDI '26

GraCE: Unlocking CUDA Graphs with Compiler Support for ML Workloads

Abhishek Ghosh, Ajay Nayak, Ashish Panwar, Arkaprava Basu

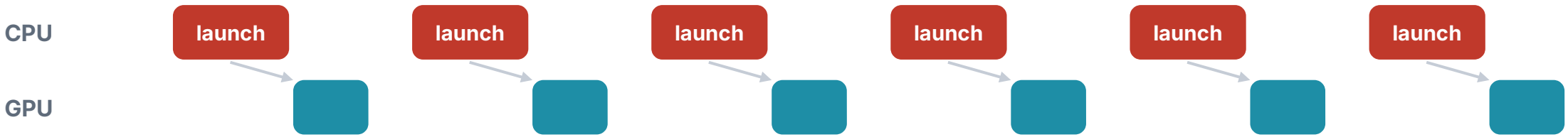
Indian Institute of Science · Microsoft Research India

20th USENIX Symposium on Operating Systems Design and Implementation · Seattle, July 13–15, 2026

MOTIVATION

The Challenge of Efficient ML Execution

Without CUDA Graphs



Repeated CPU launches: the GPU waits for the next kernel to be submitted

With CUDA Graphs



One submission; the GPU runs the captured sequence back to back

1. ML workloads run long sequences of GPU operations, many of them small kernels
2. Frequent launches make CPU-side launch overhead a significant part of execution time
3. Using CUDA Graphs efficiently is not straightforward in existing frameworks

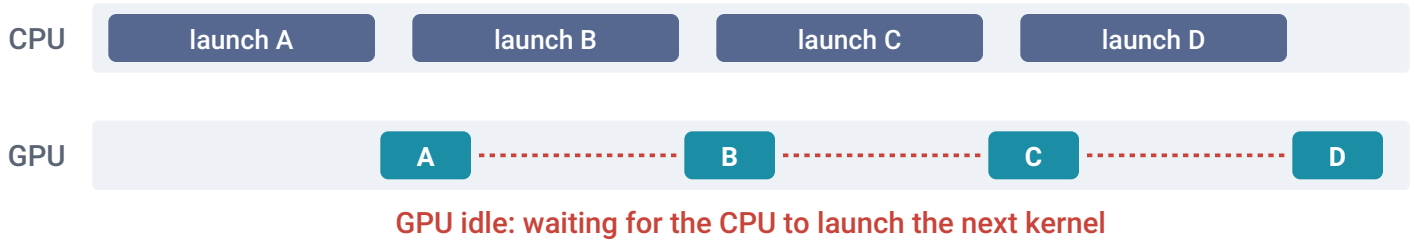
CUDA Graphs promise to remove repeated launch overhead. Next: why that matters, and how they work.

Conceptual timeline, not to scale.

Every kernel launch costs CPU time

One launch per kernel

schematic



- Models launch hundreds of short kernels per iteration
- Each launch spends about 5-10 µs on the CPU; many kernels run for only a few µs
- When the kernel is shorter than its launch, the GPU sits idle

The path of one launch



CUDA Graphs: record once, replay with one launch

1 Capture
Run once and record every kernel into a graph

→

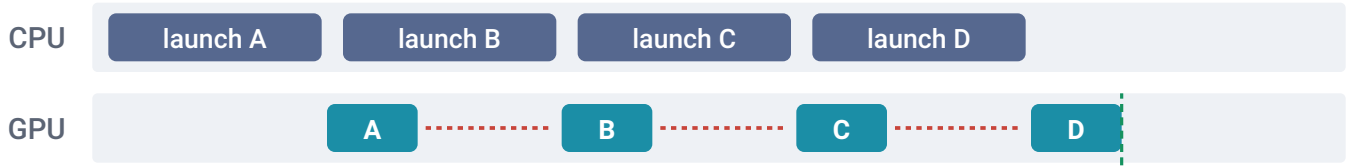
2 Instantiate
CUDA prepares all launches in advance

→

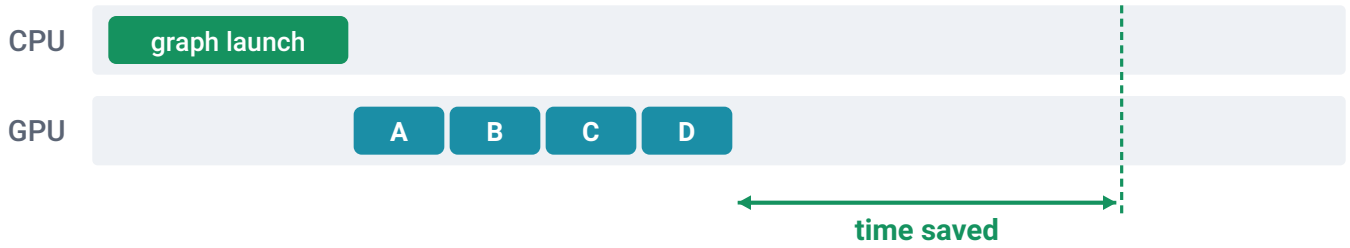
3 Replay
One cudaGraphLaunch per iteration

steps 1-2 happen once · step 3 every iteration

Without a graph



With a CUDA Graph



schematic: same kernels, same time axis

What replay skips

- Python, PyTorch dispatch and driver work run once, at capture
- Each iteration is a single cudaGraphLaunch
- The GPU runs the kernels back to back, without waiting for the CPU
- Biggest gain when there are many short kernels

The catch: every kernel argument is frozen at capture. →

INSIDE A CUDA GRAPH

Six operations become a six-node graph

The code

```
x = 4x4 matrix  # 0, 1, ..., 15

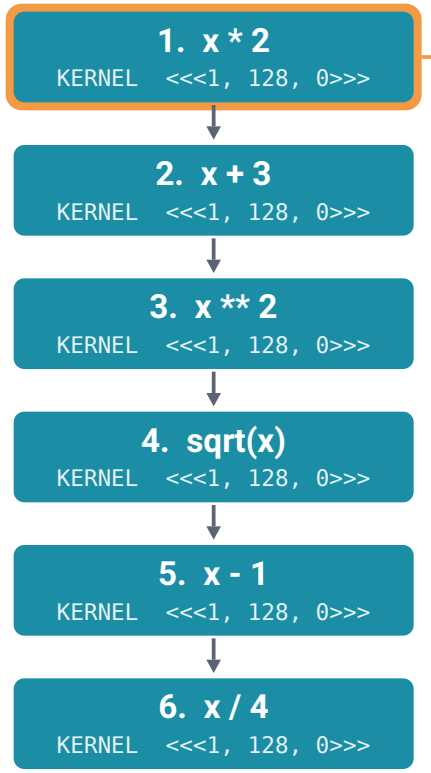
def one_iteration(x):
    x = x * 2
    x = x + 3
    x = x ** 2
    x = sqrt(x)
    x = x - 1
    x = x / 4
```

Each line is one GPU kernel and uses the result of the line above.

Captured once, then replayed every iteration. One value over 3 replays:

`0 -> 0.5 -> 0.75 -> 0.875`

The captured graph



6 nodes, 5 edges: a chain (one stream)

Node 1 up close

Type	KERNEL
Kernel	`vectorized_elementwise_kernel<MulFunctor>`
Launch	1 block x 128 threads, 0 B shared memory
Edges	none in, one out (to node 2)

Arguments, copied in at capture (32 bytes):



- The 2.0 and the address of x are frozen inside the node
- Only the data at that address changes between replays
- So the node is a complete, stored kernel launch

WHY IT IS FASTER

One launch replaces 105 launches

Without a graph: every iteration

Python op → PyTorch dispatch → driver builds launch → GPU runs kernel

repeated for each of the 105 kernels: about 2.7 µs of CPU work each

With a graph

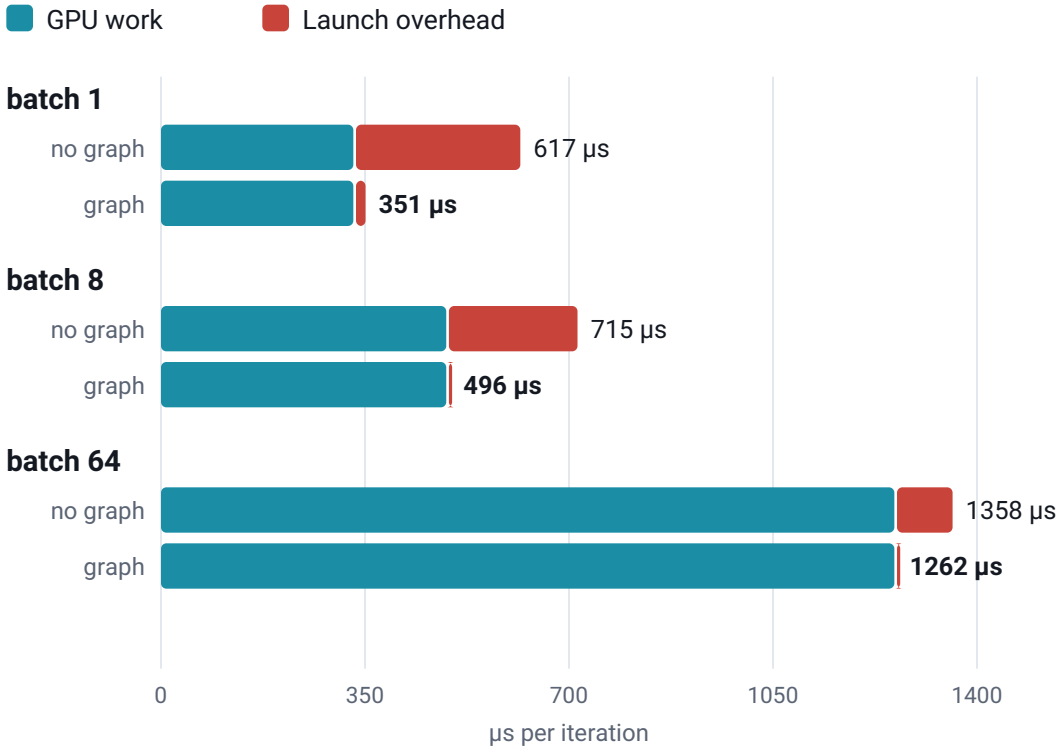
Capture (once)
the same path, but each launch is stored as a node

Every iteration
1 cudaGraphLaunch; the GPU runs all 105 nodes back to back

- Per-kernel CPU work is paid once, at capture
- Replay: no Python, no dispatch, no OS calls per kernel
- The GPU no longer waits for the CPU between kernels

Attention model: time per iteration

105 kernels; overhead = measured time minus pure GPU time



Batch 1: launch overhead drops from 287 µs to 21 µs, a 1.76x speedup

THE PROBLEMS

But CUDA Graphs are hard to use well

Every kernel argument is frozen at capture, and a graph cannot contain operations that make the CPU wait for the GPU. Studying 60+ ML and HPC workloads in PyTorch2, the paper finds three problems:

1

Code not written for CUDA Graphs

One CPU-side value, like a NumPy scalar or a CPU tensor, makes PyTorch2 skip the graph for the whole region.

Fewer kernels run inside graphs

2

Copying inputs before every replay

New input data must be copied into the graph's fixed buffers each time, and large inputs make that slow.

Every replay pays a copy

3

Graphs can make things slower

Replay has fixed costs. For small graphs they outweigh the saving, yet PyTorch2 uses graphs everywhere.

Some graphs slow down execution

Next: each problem in detail, with the paper's examples and our measurements.

PROBLEM 1 IN DETAIL

A CPU scalar makes PyTorch2 skip the CUDA Graph

```
1   import torch
2   import numpy as np
3
4   class ScaledDotProductAttention(nn.Module):
5     def __init__(self, temperature, attn_dropout):
6       ...
7       self.temperature = temperature
8       ...
9
10    def forward(self, q, k, v, mask=None):
11      attn = torch.bmm(q, k.transpose(1, 2))
12      # Divide operation below with a numpy scalar on CPU
13      attn = attn / self.temperature # offending line
14      ...
15
16  class MultiHeadAttention(nn.Module):
17    def __init__(self, n_head, d_model, d_k, d_v, dropout):
18      ...
19      temp = np.power(d_k, 0.5) # src of offending operand
20      self.attn = ScaledDotProductAttention(
21          temperature=temp, attn_dropout=dropout)
22      ...
23
24    def forward(self, q, k, v, mask=None):
25      ...
26      output, attn = self.attn(q, k, v, mask=mask)
27      ...
28
29  module = # Initialize the speech_tranformer model
30  module = torch.compile(module)
31  result = module(inputs)
```

Figure 2 (GraCE paper): Speech Transformer, simplified. Red = the offending scalar.

1. **Line 19**
   temperature is computed with NumPy, so it is a plain number on the CPU, not a GPU tensor.

2. **Line 13**
   Inside forward(), the attention scores are divided by it. The divide runs on the GPU, with temperature passed in as a kernel argument.

3. **Capture**
   A CUDA Graph would store that number inside the node. If it changed, replay would silently use the old value.

4. **PyTorch2**
   Cannot be sure it stays fixed, so for correctness it does not use a CUDA Graph for this whole region.

One NumPy scalar in one line keeps every surrounding kernel out of a graph, so the launch overhead comes back.

PROBLEM 1 IN DETAIL

CUDA captures a scalar, but freezes its value

Without a changing scalar

```
for i in range(3):
    x += 1          # new data in x
    y = x * 2 + 1
```

The loop body is captured as a CUDA Graph in iteration 1 (2 nodes):



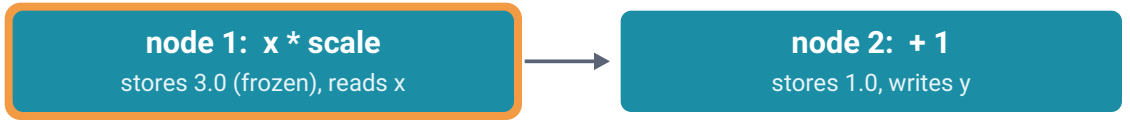
iteration	got	expected	
iteration 1	`[3, 5, 7, 9]`	`[3, 5, 7, 9]`	correct
iteration 2	`[5, 7, 9, 11]`	`[5, 7, 9, 11]`	correct
iteration 3	`[7, 9, 11, 13]`	`[7, 9, 11, 13]`	correct

New data in x is picked up: the node stores the address of x.

With a changing scalar

```
for i in range(3):
    scale = random.randint(1, 10)
    y = x * scale + 1
```

The loop body is captured as a CUDA Graph in iteration 1 (2 nodes):



iteration	got	expected	
scale = 3	`[1, 4, 7, 10]`	`[1, 4, 7, 10]`	correct
scale = 10	`[1, 4, 7, 10]`	`[1, 11, 21, 31]`	WRONG
scale = 2	`[1, 4, 7, 10]`	`[1, 3, 5, 7]`	WRONG

scale was copied into the node as 3. No error is raised.

Both graphs look the same to CUDA: a number in a node is just a number.
So PyTorch2 refuses to use a graph when a value can change, and loses the speedup.

PROBLEM 1 IN DETAIL

CUDA Graphs Impose Constraints on Memory Operations

Ordinary CUDA execution		Inside a CUDA Graph capture region
`cudaMemcpy`	synchronous copy	`cudaMemcpy` NOT PERMITTED
`cudaMalloc`	memory allocation	`cudaMalloc` NOT PERMITTED
Other APIs	that implicitly synchronize the device	**Other APIs** NOT PERMITTED
Host code issues these calls as usual		**The rest of the region is recorded** GPU operations such as kernel launches and memcopies

These restrictions force developers to rethink data movement and memory management.
Integrating CUDA Graphs into existing, legacy codebases is therefore difficult.

PROBLEM 2 IN DETAIL

Copying Inputs Before Every Replay

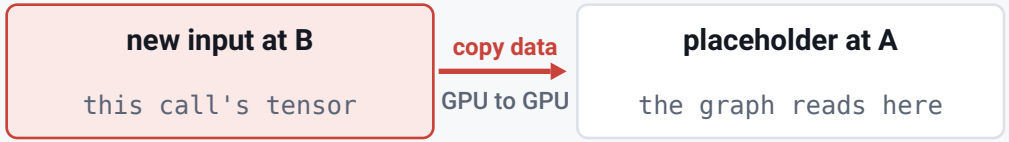
At capture: the graph stores an address



1. **One fixed buffer per input**
   PyTorch2 gives each input a placeholder in GPU memory, at address A.
2. **Kernels remember A, not the data**
   Every replay reads whatever data is at A.

Weights stay at the same address every call, so they are never copied.

Every replay: new input, new address



1. **A new tensor lands somewhere else**
   A new batch or a fresh output is allocated at a new address B.
2. **The graph only knows A**
   So PyTorch2 copies the data from B into A before each replay.

Outputs of kernels inside the graph are not copied: only inputs from outside.

One data copy per input, paid on every replay, and it grows with the input size.

PROBLEM 2 IN DETAIL

Copying Inputs: Where the Input Lives Decides

where the input is

GPU, new address B

new batch, fresh output

before replay

Copy B into A before every replay

Problem 2

where the input is

GPU, same address A

weights, data written in place

before replay

Nothing to copy: the graph reads A directly

No cost

where the input is

CPU memory

NumPy value, CPU tensor

before replay

A CPU-to-GPU copy is not allowed in a graph, so PyTorch2 skips the graph

Problem 1

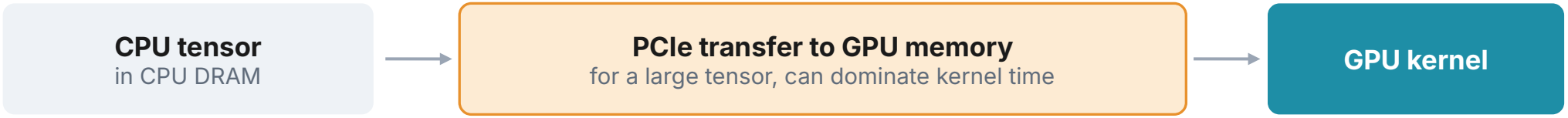
GraCE: moves CPU tensors to the GPU once (CGCT), and copies only the 8-byte address instead of the data (PI).

The graph freezes where it reads, so every input at a new address must be copied into place.

PROBLEM 1 IN DETAIL

One Unsupported Operation Can Affect an Entire Kernel Chain

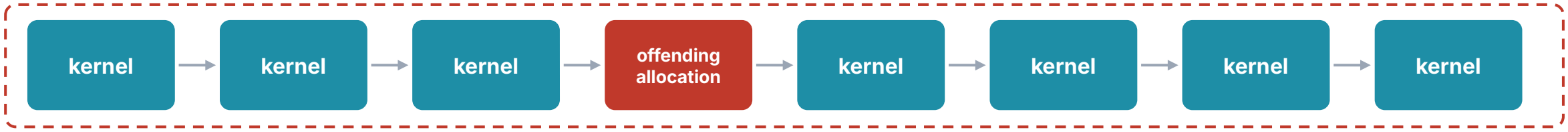
Ordinary execution



Placement can at most delay this one kernel

CUDA Graph capture

an otherwise CG-amenable, long chain of kernels



A single offending allocation can disallow any of the kernels from being captured

Ordinary execution: tensor placement can affect one kernel's latency.
CUDA Graphs: a single problematic allocation can affect the capture of a much larger kernel chain.

PROBLEM 1 IN DETAIL

Relocating Tensors Pays Off Far More with CUDA Graphs

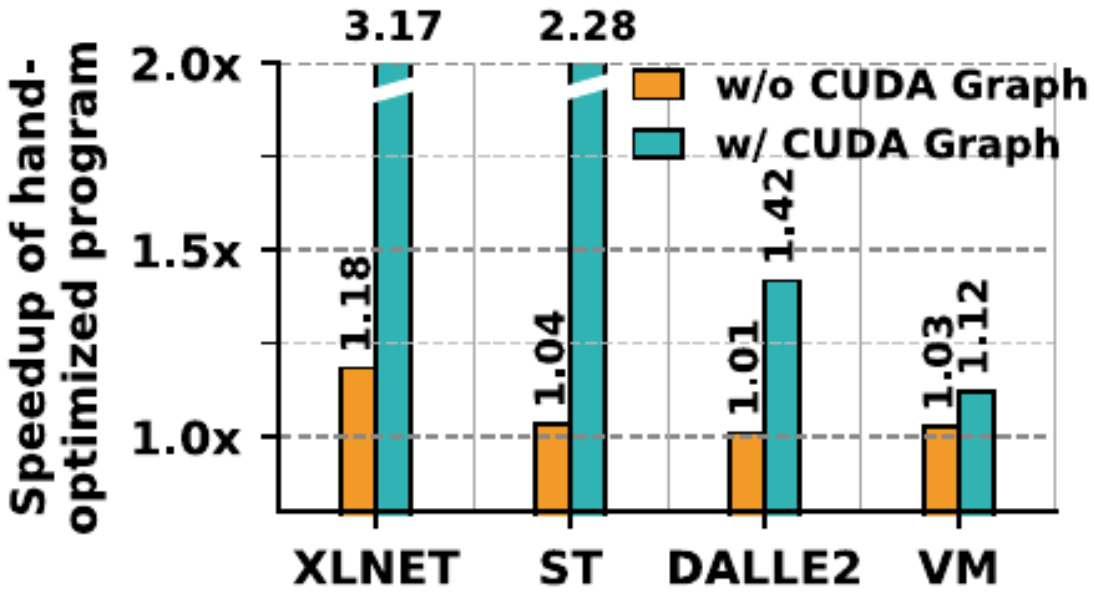


Figure 3: speedup from moving CPU-allocated tensors to the GPU. Orange: without CUDA Graph. Teal: with CUDA Graph.

Without CUDA Graphs

6.29%

average gain from optimizing tensor placement (up to 18% for XLNET)

With CUDA Graphs

3.17× · 2.28×

XLNET and ST speedups from the same kind of code edits

4-GPU tensor parallelism (XLNET, batch 128)

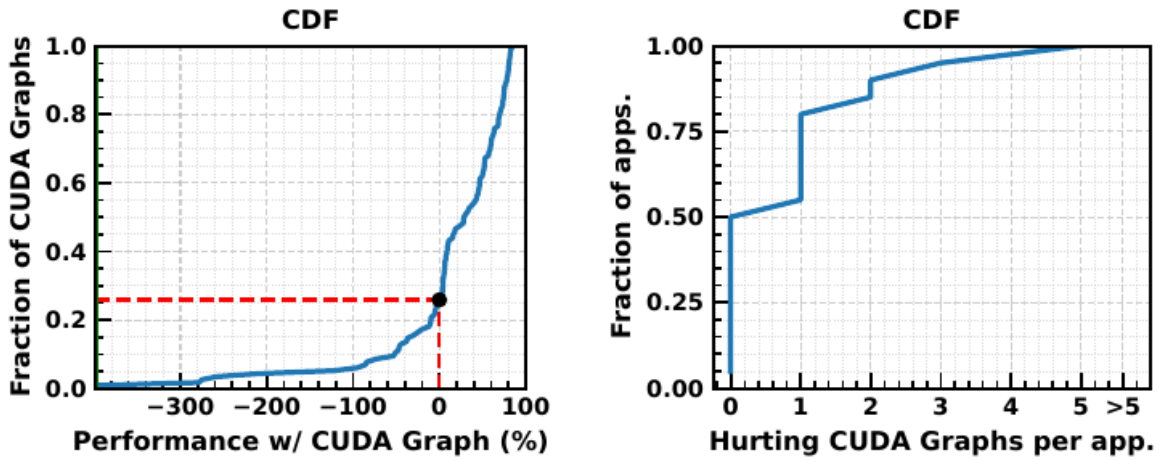
> 450 kernels

ineligible for capture due to a few CPU DRAM allocations

Same code edits: a modest gain without CUDA Graphs, a dramatic speedup with them.

Authors' manual edits to XLNET, Speech Transformer (ST), DALLE2, Vision Mask R-CNN (VM); inference, batch size 1.

How Often Do CUDA Graphs Hurt? Evidence from TorchBench



(a) 25% of the total CUDA Graphs hurt performance in TorchBench.

(b) About 50% of the applications contain CUDA Graph(s) that hurt performance.

Figure 4: Cost-benefit analysis of CUDA Graphs.

The dominant source of the slowdowns is the latency of copying kernel parameters into the static placeholders during replay.

Graph level · Figure 4a

29 of 116

CUDA Graphs (25%) degrade performance, out of all graphs analyzed

Application level · Figure 4b

≈ 50%

of applications contain hurting graphs; plot covers the 20 applications with multiple graphs

Parameter copying · §2.3.2

24% · 17%

of end-to-end execution time, in DALLE2 and NVIDIA Deep Recommender

Graph-level and application-level figures use different denominators: 116 graphs versus 20 multi-graph applications.

116 CUDA Graphs from 63 PyTorch applications in TorchBench.

TRANSITION

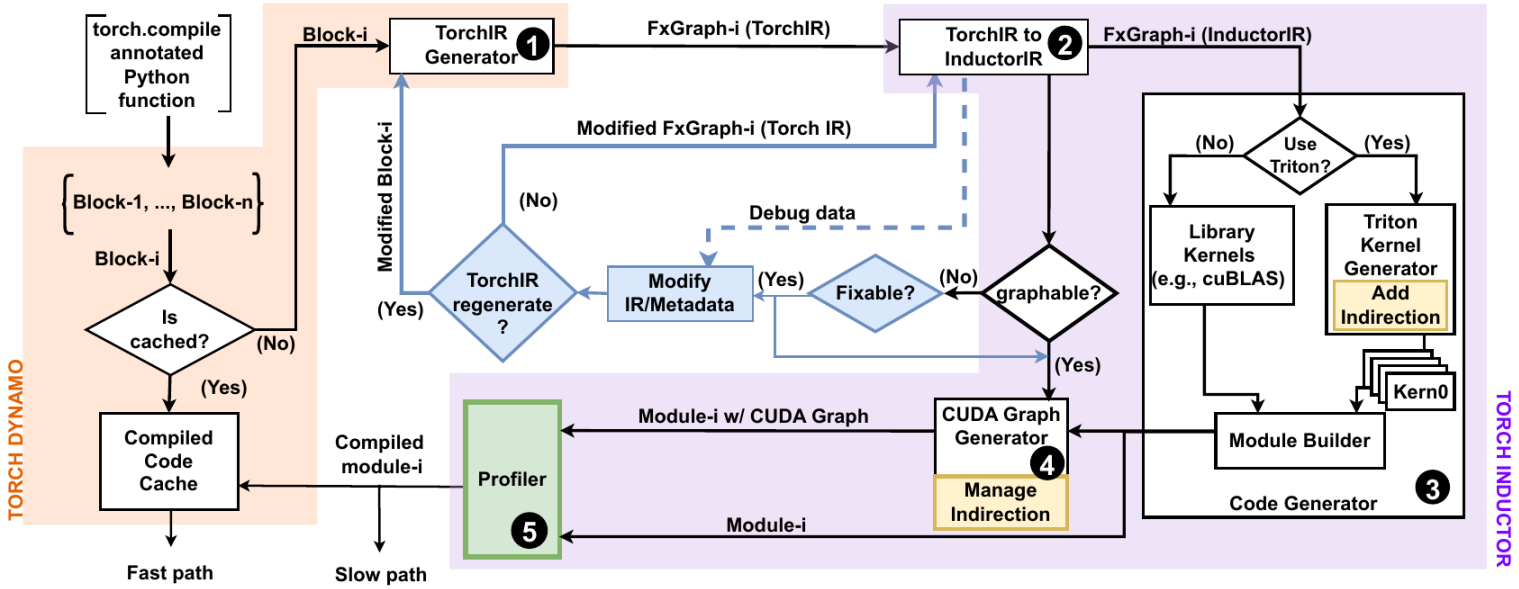
From Three Problems to GraCE's Goals

What we established	GraCE's goals
1 **Some operations prevent CUDA Graph capture** CPU-resident scalars and tensors, synchronous copies	→ **Expand the set of GPU kernels that can be captured**
2 **Parameter copying adds overhead even when capture succeeds** on the critical path of every replay	→ **Minimize runtime overhead during replay, especially parameter copy**
3 **Other replay costs and indiscriminate deployment hurt** CPU-object rebuild, RNG reset, graphs used blindly	→ **Deploy graphs only when they improve end-to-end performance**

All of this without requiring program modifications or developer intervention.

GraCE: a compiler framework built on PyTorch2 to maximize the coverage and benefits of CUDA Graphs.

How GraCE Extends PyTorch 2's Compilation Workflow



1. **Graph Construction** — TorchDynamo
2. **IR Lowering & Optimization** — TorchInductor
3. **Kernel Generation Layer** — code generator
4. **CUDA Graph Capture Layer** — CUDA Graph generator
5. **Profiler** — Selective CUDA Graphs

Figure 6: Implementation of GraCE within PyTorch2's compilation framework.

CGCT
Makes more computation graphs eligible for CUDA Graph capture.

PI
Reduces parameter-copying overhead.

SCG
Selects graph configurations based on performance.

GraCE extends the existing PyTorch 2 compilation workflow to improve CUDA Graph deployment without requiring application code modifications.

Blue: CGCT · Yellow: PI · Green: SCG.

FIGURE 6: GRAPHABLE?

Step 1: Is the Graph Graphable?

The check, already in PyTorch2

1 **Record** `graph.py`
TorchInductor walks the FX graph node by node
and notes the device of every node

2 **Decide** `cudagraph_utils.py`
Any node on the CPU, or more than one device: NO
Everything on one GPU: YES

3 **Act** `compile_fx.py`
NO: print “skipping cudagraphs due to …”
and run the region without a CUDA Graph

Example: NumPy temperature

devices recorded for the graph's inputs

input	comes from	device
arg0_1	temperature	cpu
arg1_1	scores	cuda:0

NO: skipping cudagraphs due to cpu device

One CPU input is enough: the whole region runs without a CUDA Graph.

Other NO reasons, not the focus here: .item(), changing a graph input in place.

In PyTorch2, NO ends here. GraCE asks: why NO, and can it be fixed?

FIGURE 6: DEBUG DATA

Step 2: Where Did the Problem Come From?

1. **Found in the IR, fixed in the code**
   The CPU node is found in TorchInductor's IR, but the fix has to happen where the tensor is made.

2. **Every node carries debug data**
   PyTorch2 labels each node with stack_trace (file and line) and source_fn_stack (the function).

3. **GraCE follows the labels back**
   From the IR node to its TorchIR node, and from there to the Python line or object.

Fixable? Only if the cause is a known pattern:

CPU scalar | CPU-to-GPU copy | CPU output

Anything else, e.g. code that calls .cpu() itself: not fixable, skipped as in PyTorch2.

Example: temperature

InductorIR

```
arg0_1 on cpu, used by div
```

comes from

TorchIR node

```
truediv   stack_trace: model.py:13
```

comes from

Python line

```
attn = scores / self.temperature
```

Debug data turns “this node is on the CPU” into “this line of Python made it”.

FIGURE 6: MODIFY IR/METADATA

Step 3: Modifying the IR

problem	example	what GraCE changes	where
CPU output	`idx = arange(n)`	marks the node's output device='cuda'	**the FX graph itself**
NumPy / CPU scalar	`as_tensor(temperature)`	builds the tensor with device='cuda'	**bytecode Dynamo made**
CPU weight, copied	`w = self.w.to('cuda')`	moves the weight to the GPU once	**the model's attribute**

Inside the graph	only the CPU output: the edited graph can be compiled directly.
Outside the graph	the other two: the change is in the code that builds the graph. This decides step 4.

Each fix makes the tensor live on the GPU, so no CPU node is left in the graph.

FIGURE 6: TORCHIR REGENERATE?

Step 4: Regenerate TorchIR?

No: the fix was inside the graph

e.g. a CPU output

The edited FX graph goes straight back to TorchInductor and is lowered again.

Yes: the fix was in the code feeding the graph

e.g. NumPy scalar, CPU weight

GraCE asks Dynamo to restart tracing this block. Dynamo builds a new FX graph from the fixed code.

Either way, step 1 runs again and now says YES.

The rewrite for temperature (Figure 7)

Without CGCT

```
graph_out_0 =
__compiled_fn_43(
 ..., torch.as_tensor(
  __import_torch.as_tensor(
   self.layer_stack[0]
    .slf_attn
    .attention
    .temperature
   )), ...)
```

(a) Original bytecode.

With CGCT

```
graph_out_0 =
__compiled_fn_43(
 ..., torch.as_tensor(
  __import_torch.as_tensor(
   self.layer_stack[0]
    .slf_attn
    .attention
    .temperature,
   device='cuda')), ...)
```

(b) Modified bytecode.

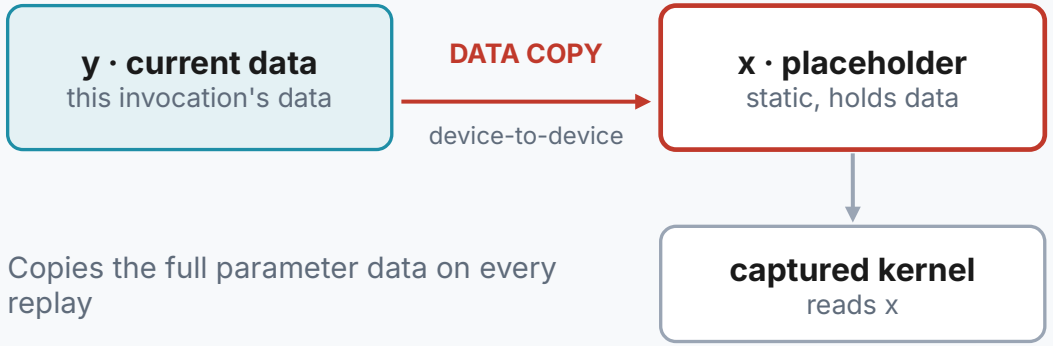
Figure 7: IR modification as part of CGCT.

Only device='cuda' is added to the bytecode Dynamo generated. The tensor is now created on the GPU.

Fix inside the graph: compile again. Fix in the code: Dynamo traces it again.

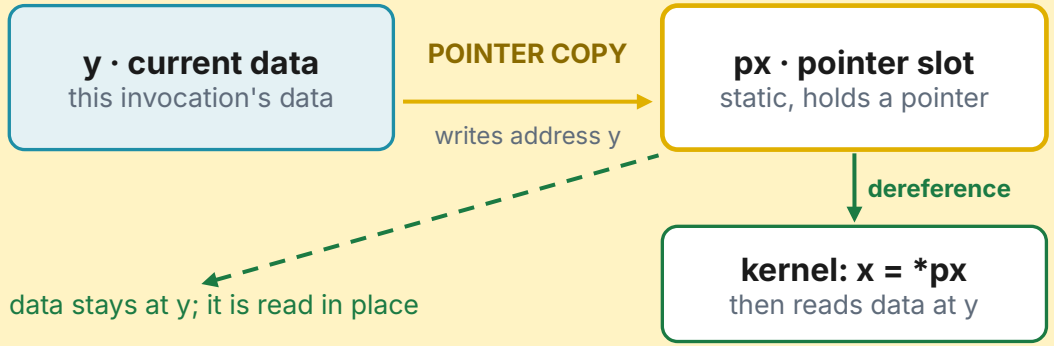
PI: Replacing Data Copies with Pointer Copies

Baseline: copy the parameter data



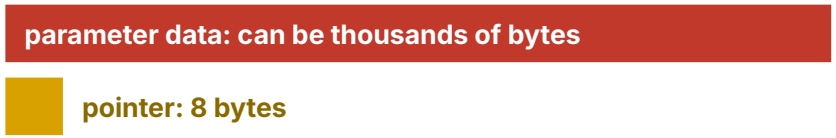
Copies the full parameter data on every replay

GraCE PI: copy the pointer



1. CUDA Graphs record kernel parameters, including tensor addresses, at capture
2. When input data changes, parameter copies may be needed before replay
3. Copying large parameter data adds overhead to every replay
4. GraCE adds a level of indirection: update the address, not the data

Per eligible parameter, per replay (conceptual, not to scale)



Data movement does not vanish: a small pointer is still copied before each replay, for eligible parameters only.

PI replaces expensive parameter-data copies with much smaller pointer updates.

Simplified redraw: x = recorded placeholder, y = current data, px = pointer-to-pointer.

How PI Updates Kernel Parameters

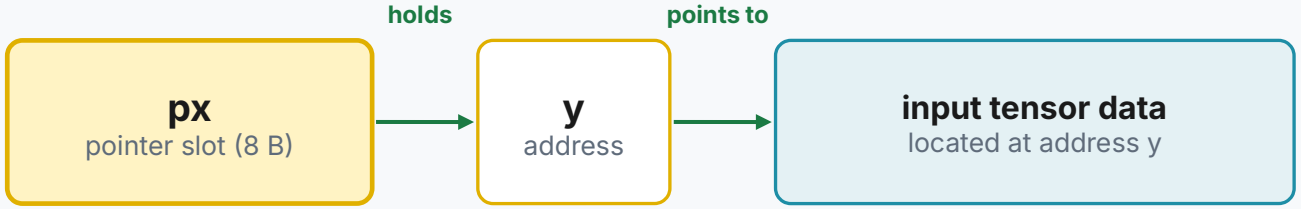
1 Capture
The graph records the kernel with its pointer-to-pointer parameter px

2 New invocation
Input data arrives at a potentially different address y

3 Update the pointer
Before replay, address y is copied into the slot px

4 Execute the kernel
The kernel dereferences px and accesses the current data

Memory view at replay



The kernel follows px → y → data, so the graph itself never changes

Baseline contrast



Only externally supplied parameters
such as application inputs. Outputs produced by earlier kernels in the same graph pass directly to later kernels, with no copy.

Graph-capture layer
Allocates placeholders for pointers instead of data, and injects pointer copies (host-to-device, on the replay's CUDA stream, so no extra synchronization).

The graph's structure stays fixed while the address of external input data changes between replays.

SLOW PATH · COMPILATION

PI for JIT Kernels: Rewrite the Kernel

Inductor generates a Triton kernel
compiled just in time, so GraCE can modify it

→

Code generation marks the parameters
attached to the triton.jit decorator

→

Custom LLVM pass rewrites the PTX
integrated with Triton

Before: pointer argument

```
__global__ void jit_kernel(int *x) {
    x[idx] = x[idx] + 1;
}
```

→

After: pointer-to-pointer argument

```
__global__ void jit_kernel(int **px) {
    int *x = *px;  // dereference
    x[idx] = x[idx] + 1;
}
```

`int *x  →  int **px`
signature change

`+ int *x = *px`
dereference inserted at kernel start

rest of the kernel unchanged
computation uses x as before

For JIT kernels, GraCE rewrites the kernel itself so it reads its data through a pointer.

Code is a simplified conceptual example, not GraCE's generated PTX.

Why Kernel Rewriting Is Not Always Possible

1 · JIT-generated kernels

Triton

Compiler-generated
during compilation

→

Rewrite kernel
pointer → pointer-to-pointer

→

Execute
rewritten kernel

- GraCE can modify the code the compiler generates
- It changes pointer arguments to pointer-to-pointer arguments and inserts dereferencing instructions

2 · Vendor-supplied kernels

e.g., cuBLAS

Precompiled binary
optimized library kernel

→

Cannot rewrite
internal code out of reach

→

GRACE'S ALTERNATIVE

Prelude kernel
updates the parameters

→

Execute original kernel
unmodified binary

- Supplied as optimized, precompiled binaries
- GraCE cannot directly rewrite their internal code, so the JIT approach is not feasible here

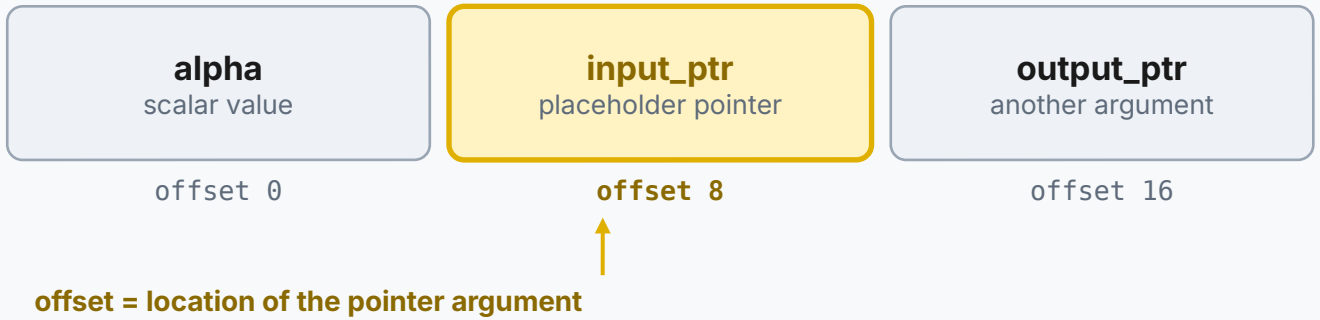
Prelude kernel, inserted before the vendor kernel in the CUDA Graph:

1. Reads the current pointer value through the pointer-to-pointer
2. Updates the vendor kernel's parameter buffer
3. Lets the original kernel run with the correct pointer

Instead of modifying the vendor kernel, GraCE inserts a prelude kernel before it in the CUDA Graph.

How Does GraCE Know Which Parameter to Update?

A vendor kernel's parameter buffer (hypothetical example)



Why the offset is needed

cudaGraphKernelNodeSetParam takes three arguments:

1 the kernel-node handle

2 a device address holding the replacement pointer value (px)

3 the offset in the node's parameter buffer

1 Obtain the buffer handle

cuFuncGetParamInfo, during compilation, gives a handle to the kernel's parameter buffer

2 Search for the placeholder pointer

byte-pattern match against the known placeholder pointers recorded during graph capture

3 Record the matching offset

the match location identifies the argument that must be updated

The offset identifies where the pointer value must be written, not where the input tensor itself resides.

The API needs an offset because it must update a specific argument in the vendor kernel's parameter buffer, not an arbitrary memory location.

Byte offsets in the diagram are illustrative, not from the paper.

From px to the Vendor Kernel: The Complete Update Path

1 **Current pointer y**
invocation's input tensor address

pointer copy before replay (H2D, not the tensor)

2 **Pointer slot px**
device memory, recorded at capture

prelude dereferences px

3 **Current pointer value y**
obtained by *px

device-side API writes at the recorded offset

3 **Vendor kernel parameter buffer**
field at the identified offset

vendor kernel executes

4 **Access current input data at y**
unmodified vendor kernel runs

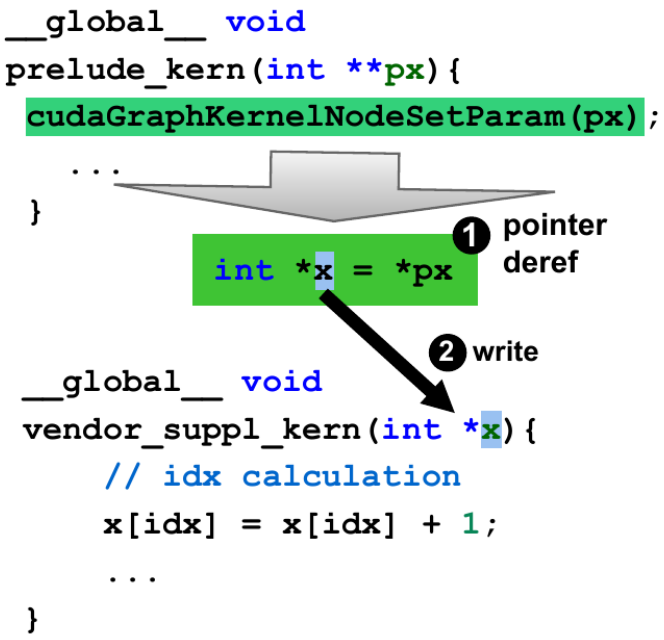


Figure 8: Indirection in vendor-supplied kernels.

In the figure

1 pointer deref

2 write

x is the parameter being updated

Two different locations: px is the device-memory slot holding the replacement pointer; the offset says where that pointer is written in the vendor kernel's parameter buffer.

The prelude reads the current pointer from px and writes it into the vendor kernel's argument slot, so the unmodified kernel runs with the current address.

Two Implementations, One Goal

Aspect	JIT kernel rewriting	Vendor-kernel prelude
Target	Kernels generated through Triton	Precompiled vendor kernels
Technique	Rewrite kernel arguments; insert dereferencing instructions	Insert a prelude kernel that updates another node's parameters
Update happens	Inside the rewritten kernel	In a separate kernel before the vendor kernel
Advantage	No separate prelude kernel; lower latency (Fig. 9)	Works for kernels GraCE cannot rewrite
Limitation	Needs control over kernel generation and compilation	Extra prelude-kernel overhead; scales poorly

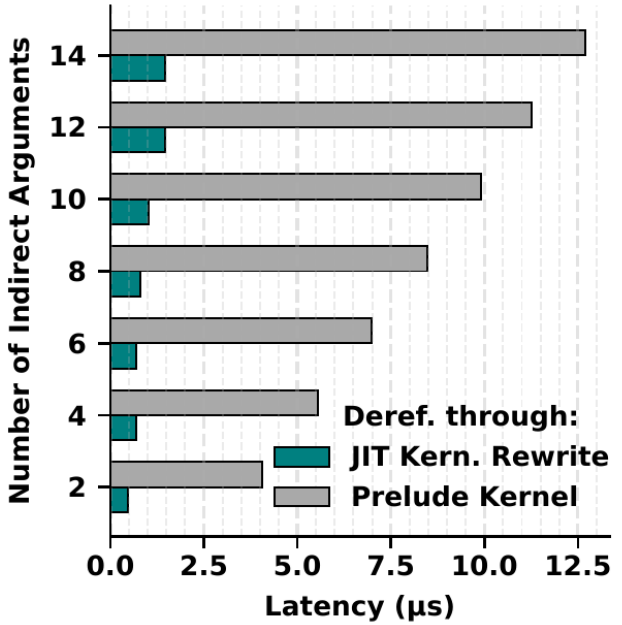
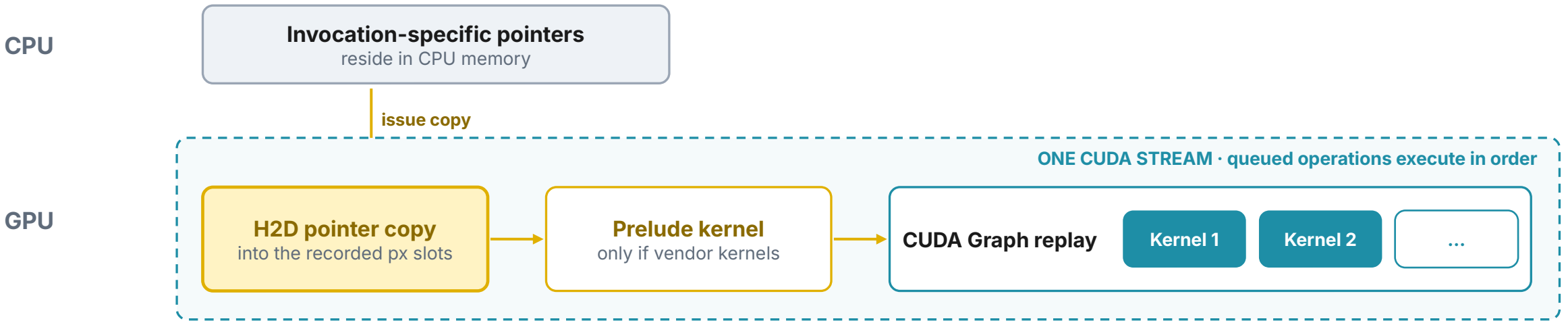


Figure 9: Latency comparison of PI approaches.

In the paper's measurements, the prelude approach is substantially slower and scales poorly, exceeding 10 µs at higher parameter counts.

GraCE uses direct kernel rewriting where possible and falls back to prelude kernels for vendor-supplied kernels.

How Does GraCE Update Pointers Before Replay?



1 Prepare
the current, invocation-specific pointers reside on the CPU

2 Copy to the GPU
pointer values go into the recorded device-memory slots

3 Same CUDA stream
the host-to-device copies use the stream of the subsequent replay

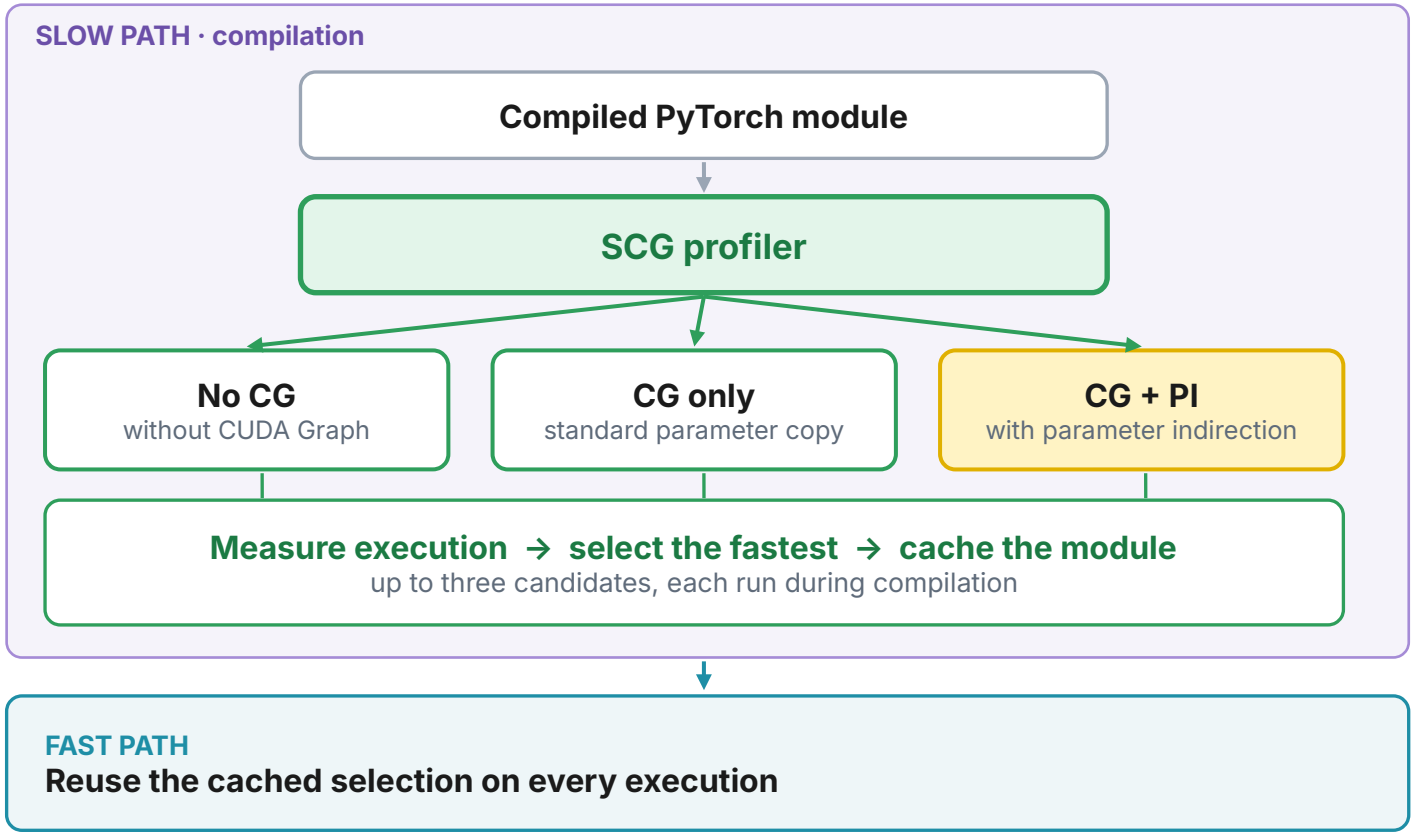
4 Execute in order
queued operations run sequentially, so updates precede use

Same-stream ordering ensures correctness without an additional host-device synchronization.
The pointer copy still takes time; the claim is only that no extra synchronization step is needed.

GraCE updates the pointers needed for replay while preserving execution order, with no separate synchronization step.

Timeline is conceptual: no durations are implied.

Selective CUDA Graphs: Profile, Compare, Select



- PyTorch 2 deploys CUDA Graphs greedily wherever capture is possible
- GraCE considers up to three configurations for each compiled module
- The profiler runs them in the slow path, measures their times, compares, and selects the best
- The result is cached for the fast path

Three candidates evaluated during compilation
not three alternatives evaluated on every replay. The decision is made independently for each CUDA Graph.

SCG decides CUDA Graph deployment from measured performance, not from the assumption that every capturable graph is beneficial.

Why Isn't CUDA Graph + PI Always the Best Choice?

Configuration	Parameter-update mechanism	Potential advantage	Potential cost
CUDA Graph without PI	Copy parameter data device-to-device, within GPU memory	Avoids pointer transfers over PCIe	Can move large amounts of data
CUDA Graph with PI	Copy pointer values host-to-device, over PCIe	Moves much less data per eligible parameter	Pointer transfers and indirection add overhead

Many small parameter objects
Pointer transfers over PCIe can cost more than copying the original data inside GPU memory.

Hardware varies
GPU memory bandwidth and PCIe transfer bandwidth differ across systems, so PI's benefit varies.

The profiler measures both CUDA Graph configurations on the target hardware and chooses the faster one for the workload.

PI reduces data movement, but the fastest choice depends on GPU-memory copies, PCIe transfers and other replay overheads.

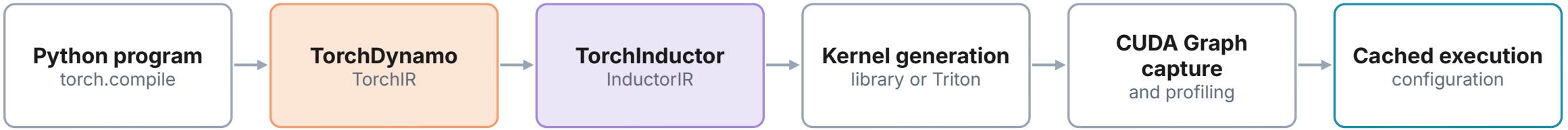
The paper motivates SCG by this variability; it does not claim PI always or never wins.

IMPLEMENTATION · SUMMARY

From Python Program to Optimized Execution

SLOW PATH · compilation and profiling

FAST PATH



CGCT
Expands capture eligibility

Analyzes and fixes supported capture obstacles

PI
Reduces replay overhead

Reduces applicable parameter-copying overhead

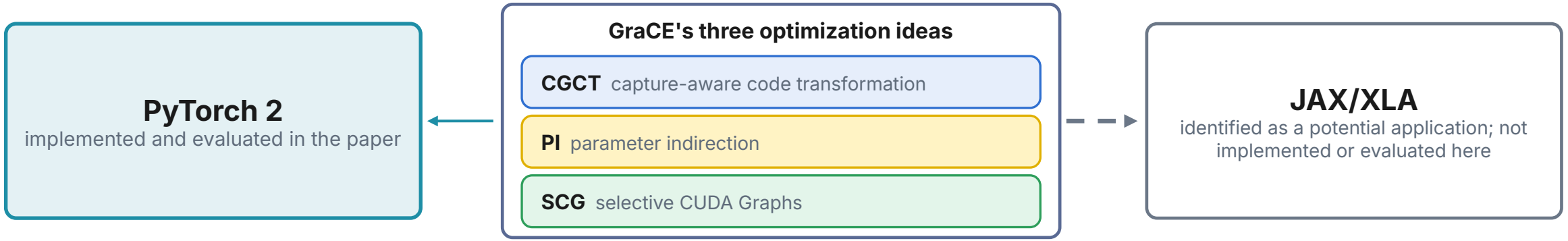
SCG
Chooses when graphs pay off

Selects deployment from measured performance

Integrated into PyTorch 2's compilation workflow; applied without manual changes to the application.

GraCE captures more useful work, reduces replay overhead, and avoids indiscriminate CUDA Graph deployment.

GraCE's Optimizations Extend Beyond PyTorch 2



Capabilities that support the techniques

- **Compiler IR with metadata**
- **Graph capture**
- **JIT kernel generation**

CGCT
Uses compiler information to find and transform capture obstacles. Suits frameworks with suitable IRs and compiler support.

PI
Replaces eligible data copies with pointer copies. Suits frameworks with graph capture and JIT kernel generation.

SCG
Profiles alternative configurations and picks the best for the target hardware.

GraCE shows a compiler-driven way to make CUDA Graphs more usable and efficient; the ideas may extend to frameworks with similar mechanisms.

JAX/XLA is the paper's stated potential, not an implemented port.

Evaluation: Objectives, Workloads & Environment

Evaluation questions

1 Overall performance
How well does GraCE exploit CUDA Graphs relative to the widely used PyTorch2 compiler?

2 Optimization impact
How much does each optimization in GraCE contribute to end-to-end performance?

3 Generalization
How robustly does GraCE generalize to distributed models and diverse GPUs?

Workloads & metric

- Sources: TorchBench, HuggingFace and TIMM
- Training and inference across vision, NLP and HPC, chosen for sensitivity to CUDA Graph usage
- **Primary metric: per-iteration runtime**
- Inference iteration = one forward pass; training = forward + backward
- Averaged over 100 runs, each with a different input sample at the workload's configured batch size

Environment (the paper's experimental setup)

GPU	NVIDIA H100 NVL, 94 GB HBM3	**PyTorch**	2.4
CPU	64-core Intel Xeon 8462Y+	**CUDA**	12.8
System memory	512 GB DDR5	**cuDNN**	8.9.2
Multi-GPU runs	four 80 GB H100s, NVLink, CUDA 12.4	**NVIDIA driver**	575.57.08

Every comparison measures per-iteration runtime, averaged over 100 runs with different input samples.

Environment is the paper's experimental setup.

Evaluation Workloads and Baselines

TorchBench 14 applications

Application	Short name	Type	Batch size
Equation of State	**EOS**	**Inference**	1048576
Speech Transformer	**ST**	**Inference**	1
DALLE2_pytorch	**DALLE2**	**Inference**	1
Multi-modal Clip	**MMC**	**Inference**	1
Turbulent Kinetic Energy	**TKE**	**Inference**	1048576
Deep Recommender	**DR-T**	**Training**	256
Deep Recommender	**DR-I**	**Inference**	8192
Vision MaskRCNN	**VM**	**Inference**	1
AlexNet	**ALNET**	**Inference**	2
dlrm	**DLRM**	**Inference**	2048
TIMM Efficientdet	**TEFD**	**Training**	1
dcgan	**DCGAN**	**Training**	32
BERT	**BERT**	**Training**	16
densenet121	**DNET**	**Inference**	64

HuggingFace 10 applications

Application	Short name	Type	Batch size
XLNet LM-head Model	**XLNet-I**	**Inference**	1
XLNet LM-head Model	**XLNet-T**	**Training**	8
MT5ForConditionalGeneration	**MTCG-T**	**Training**	16
MT5ForConditionalGeneration	**MTCG-I**	**Inference**	16
MobileBertForQuestionAnswering	**MBERT**	**Inference**	128
DebertaV2ForQuestionAnswering	**DBV2**	**Inference**	1
BlenderbotSmallForConditionalGeneration	**BSCG**	**Training**	64
DistillGPT2	**DGPT2**	**Training**	16
Speech2Text2ForCausalLM	**STCLM**	**Training**	256
DebertaForMaskedLM	**DMLM**	**Training**	8

TIMM 1 application

lcnet_050	**LCNET**	**Inference**	256

Baselines

PyTorch2-No-CG
PyTorch 2 without CUDA Graphs

PyTorch2-CG
PyTorch 2 with CUDA Graphs enabled

All compilers are based on PyTorch v2.4

T: Training, I: Inference.

Figure 10: CUDA Graphs Can Hurt Performance

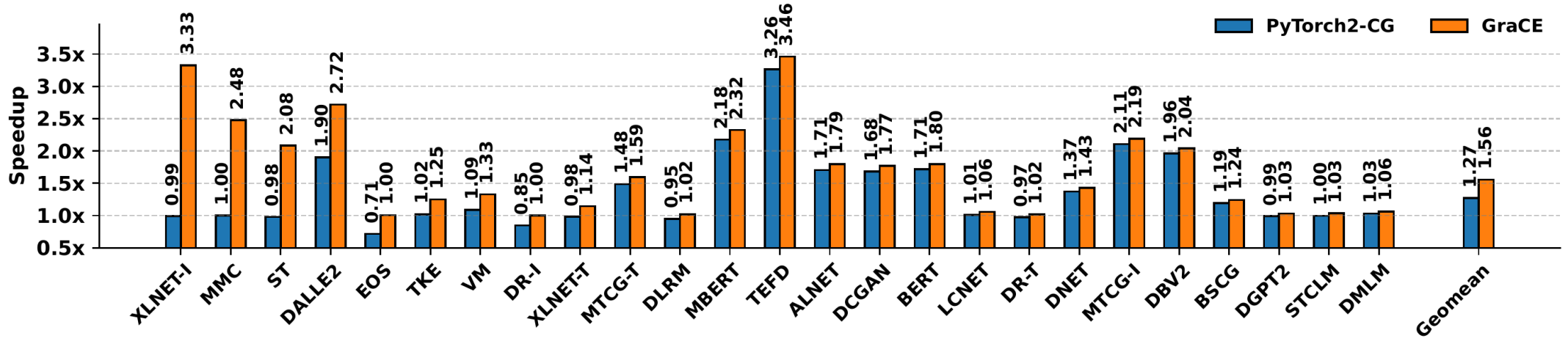


Figure 10: Performance improvement of PyTorch2-CG and GraCE against PyTorch2-No-CG.

Speedup over PyTorch2-No-CG (baseline = 1×)

Performance regressions

PyTorch2-CG slows down 4 of 25 applications by at least 3% vs. PyTorch2-No-CG.

Why it happens

For kernels lasting tens of µs, replay overhead (parameter copying, RNG management) can dominate, often ~50% of execution time.

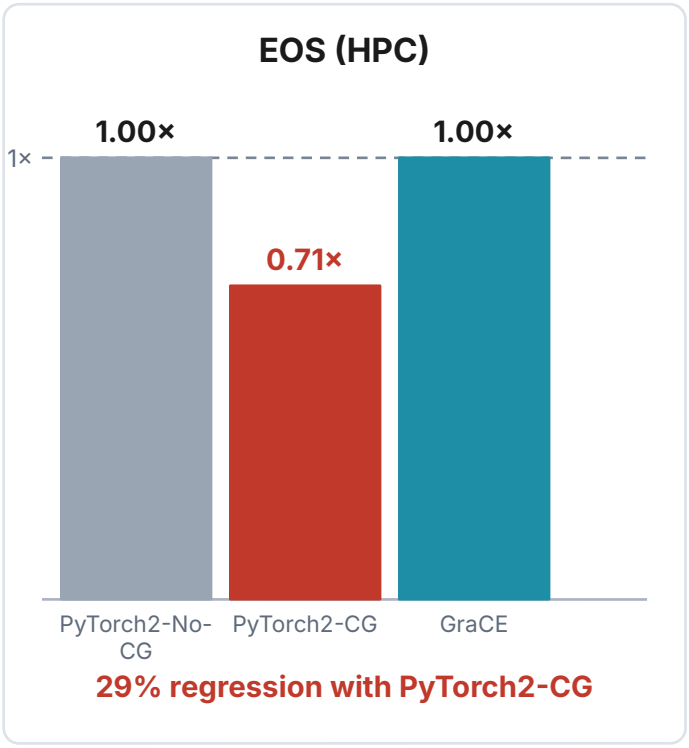
The limitation

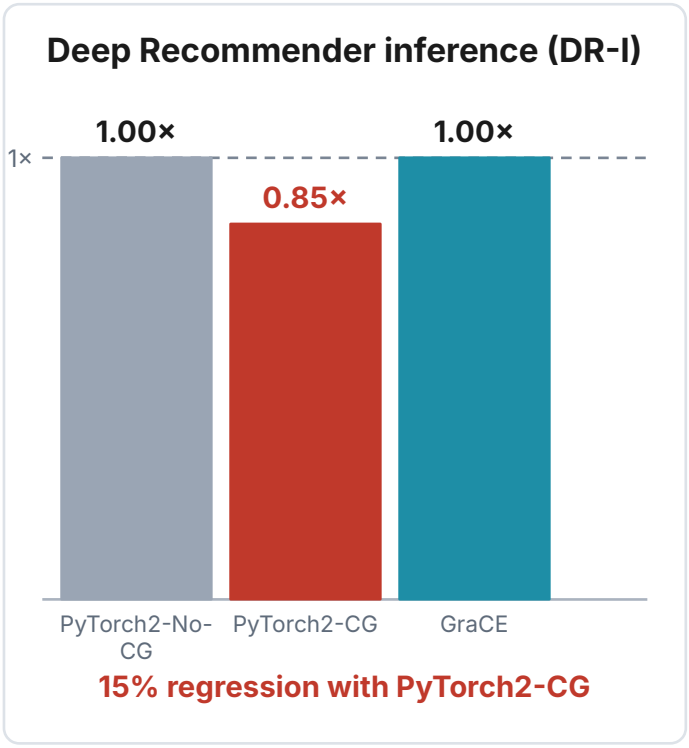
PyTorch2-CG deploys CUDA Graphs even when their overhead outweighs the benefit.

Next: Which applications suffer the most, and how does GraCE avoid these regressions?

Where Standard CUDA Graphs Regress

Speedup over PyTorch2-No-CG (baseline = 1×)





Why CUDA Graphs hurt here

Both workloads consist of short kernels, tens of µs each. Replay overhead (parameter copying, RNG management) then takes a large share of execution time, so cutting CPU launch overhead is not always enough.

GraCE's advantage

- At least as good as the better of PyTorch2-No-CG and PyTorch2-CG for every evaluated application.
- Selective CUDA Graphs (SCG) chooses the better-performing configuration instead of blindly enabling CUDA Graphs.

GraCE removes the need for users to decide, application by application, whether to enable CUDA Graphs.

Bars redrawn from the values printed in Figure 10.

Ablation of the Individual Optimizations

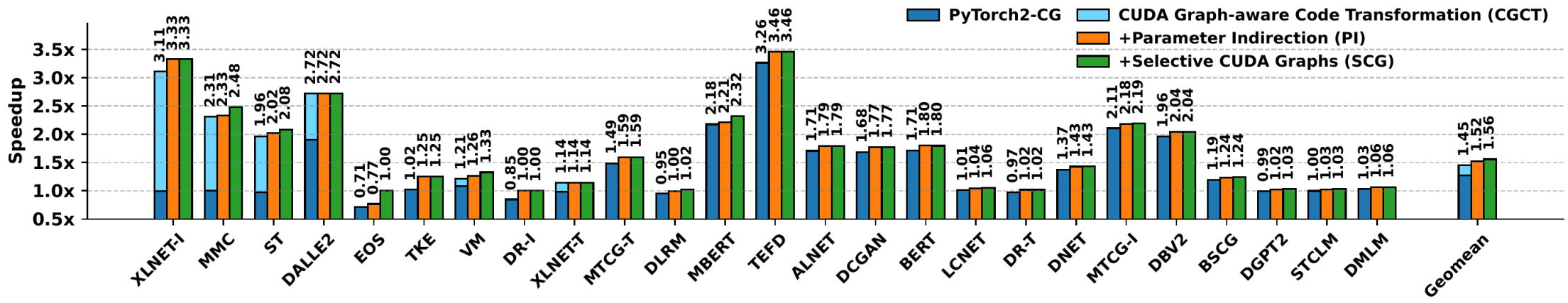


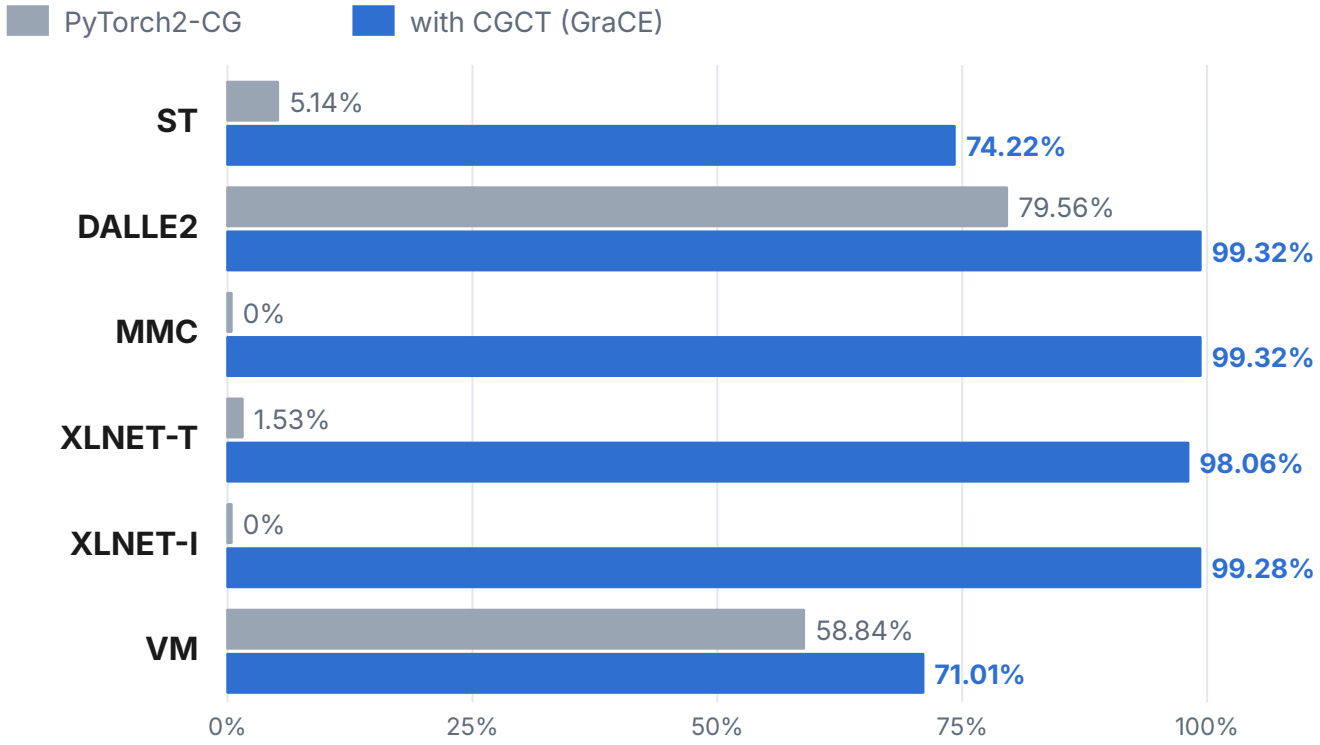
Figure 11: Ablation study on different optimizations of GraCE (performance normalized against PyTorch2-No-CG).

- Bar height is the speedup normalized to PyTorch2-No-CG.
- First bar (CGCT alone): total height is the speedup from CUDA Graph-aware Code Transformation. Dark blue (bottom) is PyTorch2-CG over PyTorch2-No-CG; light blue (top) is the extra gain from CGCT over PyTorch2-CG.
- Second bar (+PI): performance of Parameter Indirection in tandem with CGCT.
- Third bar (+SCG): cumulative effect of all three optimizations, with Selective CUDA Graphs.

CGCT

CGCT: More Kernels Run Inside CUDA Graphs

% of kernels executed inside CUDA Graphs



What blocked capture in PyTorch2-CG

XLNET-I

FX graph with 413 kernels not captured: no CUDA Graph at all. GraCE runs 413 kernels through CUDA Graphs.

MMC

PyTorch2-CG fails to deploy CUDA Graphs; GraCE enables graph capture.

DALLE2 (inference)

A single CPU-to-GPU memcpy blocks capture of an FX graph with 314 kernels.

CGCT removes the capture constraints: coverage reaches about 99% for XLNET-I, MMC and DALLE2.
ST (5.14% → 74.22%) and VM (58.84% → 71.01%) also gain substantially.

PI

Parameter Indirection (PI): Experimental Results

App.	Bytes Copy Reduced (# CUDA Graphs)	App.	Bytes Copy Reduced (# CUDA Graphs)
XLNET-I	8.0 KB → 16.0 B (1)	MMC	314.0 KB → 16.0 B (1)
ST	1.3 MB → 336.0 B (3)	DALLE2	4.0 KB → 16.0 B (5)
EOS	4.1 MB → 24.0 B (0)	TKE	174.0 MB → 168.0 B (1)
VM	1.6 MB → 184.0 B (3)	DR-I	3.0 GB → 8.0 B (1)
XLNET-T	362.0 MB → 232.0 B (3)	MTCG-T	1.0 GB → 312.0 B (3)
DLRM	14.6 MB → 136.0 B (1)	MBERT	130.0 KB → 24.0 B (1)
TEFD	6.2 MB → 120.0 B (2)	ALNET	588.0 KB → 8.0 B (1)
DCGAN	1.5 MB → 16.0 B (1)	BERT	156.3 MB → 32.0 B (2)
LCNET	73.5 MB → 8.0 B (1)	DR-T	290.0 MB → 16.0 B (2)
DNET	18.4 MB → 8.0 B (1)	MTCG-I	32.0 KB → 16.0 B (1)
DBV2	4.0 KB → 24.0 B (1)	BSCG	1.6 GB → 56.0 B (30)
DGPT2	953.4 MB → 136.0 B (3)	STCLM	849.5 MB → 136.0 B (14)
DMLM	548.8 MB → 136.0 B (3)		

Table 3: Reduction in data copy and number of CUDA Graphs optimized by GraCE's parameter indirection.

Largest speedups from PI

23% TKE **18%** DR-I

12 more workloads: speedup > 4%

EOS | XLNET-I | MTCG-T | TEFD
DCGAN | DLRM | BERT | DR-T
ALNET | DNET | BSCG | VM

Limitation: PI does not always help

DALLE2 sees no benefit: for a small tensor, the pointer-copy overhead can outweigh the savings from avoiding a larger copy within GPU HBM.

SCG: Cost-Benefit-Driven Graph Deployment

SCG enables or disables each candidate CUDA Graph based on its performance impact, so graphs are never deployed blindly.

EOS 1 candidate graph

1 candidate → disabled

Disables its single graph, avoiding a 29% performance degradation.

VM 21 candidate graphs

21 candidates → 4 enabled, 17 disabled

Enables only 4 and disables 17, improving end-to-end performance by 6%.

Overall results

5 applications
where GraCE selectively deploys graphs

97 of 123 candidate CUDA Graphs enabled



SCG avoids performance regressions by deploying CUDA Graphs only when they provide a benefit.

GraCE Under Tensor Parallelism and on a Different GPU

Speedup over PyTorch2-No-CG at each TP setting

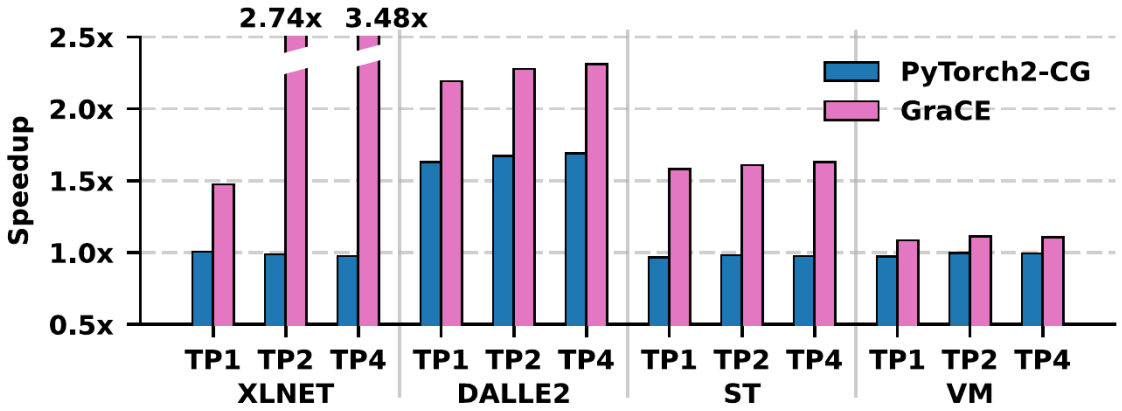


Figure 12: Speedup of PyTorch2-CG and GraCE over PyTorch2-No-CG across tensor parallel settings (TP=1,2,4).

Experimental setup

- Distributed evaluation: 4 × NVIDIA H100 GPUs connected via NVLink.
- Inference batch size 1, except XLNET (128) and DALLE2 (2).
- PyTorch2-No-CG, PyTorch2-CG and GraCE compared under TP-1, TP-2 and TP-4.

Additional evaluation: NVIDIA A6000

- Workstation-class GPU; PyTorch2-CG degrades EOS by up to 32% vs. PyTorch2-No-CG.
- GraCE never degrades performance.

3.25×
max over PyTorch2-No-CG

1.18×
avg over PyTorch2-No-CG
(PyTorch2-CG: 1.06×)

Analyzing Overheads of GraCE

Compilation-time overhead, normalized to PyTorch2-CG

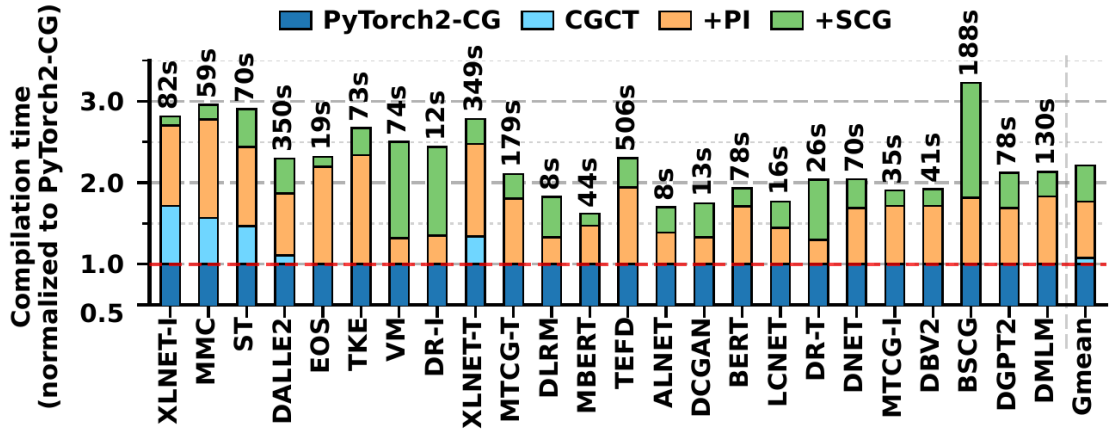


Figure 13: Breakdown of compilation time overheads of GraCE, normalized to PyTorch2-CG. The stacked bars show the cumulative contribution of CGCT, PI, and SCG. Numbers above the bars show absolute compilation time in seconds.

Compilation time (slow path)

2.21× average increase; max 506 s (TEFD)

- Figure 13: cumulative overhead of CGCT, PI and SCG.
- One-time cost per application and hardware platform, amortized over repeated graph replays.

Memory footprint

During compilation: peak memory +12% on average. SCG profiles several graph-deployment choices and temporarily retains captured tensors and allocation metadata.

During replay: no added memory; peak footprint reduced by up to 15% vs. PyTorch2-CG.

CONCLUSION

GraCE Makes CUDA Graphs Practical and Efficient

CGCT

Improve graph coverage

Transforms kernel execution patterns to enable CUDA Graph capture for more workloads.

PI

Reduce parameter-copy overhead

Uses parameter indirection to avoid repeatedly copying large parameter data.

SCG

Avoid performance regressions

Selectively enables CUDA Graphs only when they are beneficial.

Key takeaway

GraCE combines compiler-guided transformations, efficient parameter updates, and selective graph deployment to reduce GPU launch overhead while avoiding cases where CUDA Graphs hurt performance.

29% **average performance improvement over PyTorch2-CG in the evaluated workloads**